

Conv_Simple — UDFB Intro

Build the simplest CCW user-defined function block that controls the GS10 VFD: Green PB seal-in + 3-position direction switch + e-stop

Audience: First-time UDFB author who can already read CCW Structured Text | **PLC:** Allen-Bradley Micro 820 (2080-LC20-20QBB)

Drive: AutomationDirect GS10 DURApulse, Modbus slave node 1, controlled via `vfd_cmd_word`

Project: `MIRA_v5` | **IDE:** Connected Components Workbench (CCW) v22+

Primary reference: *GS10 DURApulse User Manual* + `plc/GS10_Integration_Guide.md`

This doc supersedes: nothing — new lesson, but assumes the cmd-word fixes from `Conv_Simple_Prog3_Modbus_Polling.pdf` are already in place

You will understand after reading: how to build a sealed-cabinet UDFB for motor control, why every variable lives in the scope it's in, and what changes in Prog2 ST so the new UDFB owns `vfd_cmd_word` outright.

❶ One-page cheat sheet — the values + I/O that absolutely must be right		
What	Value / Tag	Source
Green pushbutton (NO momentary, illuminated)	<code>_IO_EM_DI_04</code> (PBRun)	Deployed Prog2 ST; verified on bench
Direction selector FWD position (NO contact)	<code>_IO_EM_DI_00</code> → decoded as <code>dir_fwd</code>	Deployed Prog2 ST
Direction selector REV position (NO contact)	<code>_IO_EM_DI_01</code> → decoded as <code>dir_rev</code>	Deployed Prog2 ST
E-stop dual-channel (NC + NO)	<code>_IO_EM_DI_02</code> + <code>_IO_EM_DI_03</code> → decoded as <code>e_stop_active</code> + <code>estop_wiring_fault</code>	Deployed Prog2 ST
Green run lamp (motor running indicator)	<code>_IO_EM_DO_00</code> (LightGreen)	Deployed Prog2 ST
VFD command global (UDFB writes here)	<code>vfd_cmd_word</code> : INT	Deployed; shipped to drive by Prog3-Modbus FC06 write at <code>0x2000</code>
Cmd word STOP	1	GS10 manual p.6
Cmd word FWD + RUN (deployed value; bench-verify)	18 (<code>0x0012</code>)	<code>GS10_Integration_Guide.md</code> §2; bit-math contradiction open — see <code>Conv_Simple_Prog3_Modbus_Polling.pdf</code> §15

What	Value / Tag	Source
Cmd word REV + RUN (deployed value; bench-verify)	20 (0x0014)	Same — bench-verify before relying on production
UDFB name (this guide)	motor_starter	this doc
UDFB instance name (this guide)	motor_starter_inst	this doc
UDFB inputs (in declared order)	Enable , StartPB , DirFwdSw , DirRevSw , EStopOK	this doc §6/§7
UDFB outputs (in declared order)	VFDCmdWord : INT , RunLamp : BOOL	this doc §6/§7
UDFB locals	RunLatch : BOOL , DirLatch : INT , PrevStartPB : BOOL	this doc §7

⚠ Carry-forward from Conv_Simple_Prog3_Modbus_Polling.pdf — these must be true before this guide works

1. **Channel := 2** in all four *_local_cfg structs in Prog2 ST. Embedded RS-485 is Channel 2 on the Micro 820, not 0.
2. **ElementCnt := 4** on the monitor read in Prog2 ST. Reading 6 lands in undocumented register 0x2108 .
3. **FWD/REV command word values bench-verified.** This guide uses 18 / 20 because that's what the deployed code uses, but the bit-math across the three sources contradicts (18 vs 20 vs 34). Confirm the working value on your bench before relying on this UDFB to spin in the correct direction. The §10 test procedure includes a direction-check test that catches this.

1 The Big Picture

You're going to push a green button. A motor will spin. Whether the motor spins forward or reverse depends on where a 3-position selector switch is pointing when you press the button. Flipping the selector while the motor is running will stop it — you have to press the button again to start it back up in the new direction. Pressing the e-stop kills it immediately. Releasing the e-stop alone does not restart the motor; you have to press the green button again.

You're going to build that behavior **once**, inside a sealed box of logic called a User-Defined Function Block (UDFB). The sealed box has labeled terminals on its outside: one for the green button, one for each direction-switch position, one for the e-stop status, one for "enable", and on the output side one terminal that drops out the VFD command word and one that lights the run lamp. Inside the box, the logic is the same every time. You wire one box up to one motor. If you ever need a second motor, you wire a second copy of the same box up to a second pushbutton + selector + lamp. Fixing a bug inside the box fixes both motors at once.

2 The Real-World Problem

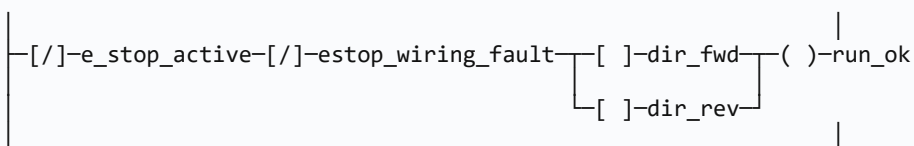
The bench conveyor has one motor today: a 2-HP 3-phase induction motor wired through the GS10 VFD. The control panel has a green illuminated pushbutton, a 3-position selector switch (FWD / OFF / REV), and a mushroom e-stop. Today, the motor-control logic lives inline inside Prog2 ST as a 5-state machine (IDLE → STARTING → RUNNING → STOPPING → FAULT). It works. But it's a wall of `CASE` statements that all reference the same global tags, and the moment a second motor needs the same treatment, you're copy-pasting the entire state machine and search-and-replacing tag names.

This guide does two things at once: (1) teaches the canonical seal-in motor-starter pattern as a UDFB, and (2) refactors the deployed Prog2 ST so the new UDFB is the sole authority on `vfd_cmd_word` — no more split-brain between the state machine and a new block fighting over the same global.

3 The Simple Version First

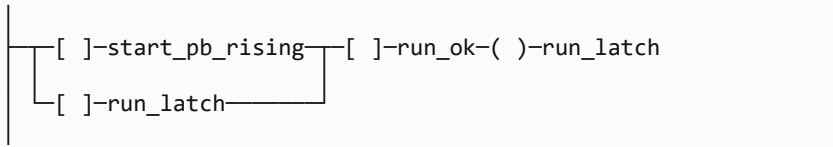
Before the UDFB exists, the simplest correct version of this logic is five flat ladder rungs in Prog3 (or in Prog2 ST). Show them flat first, prove they work, then go look at the pain.

Rung 1 — Drop the seal-in on any stop condition



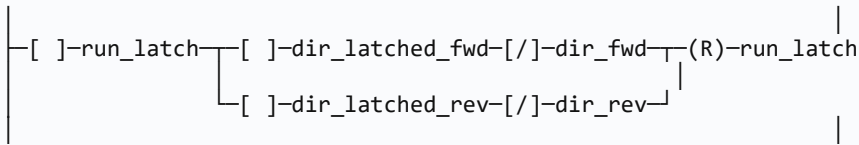
An "OK to run" composite bit. TRUE means e-stop healthy AND a direction is selected.

Rung 2 — Seal-in latch on the green pushbutton



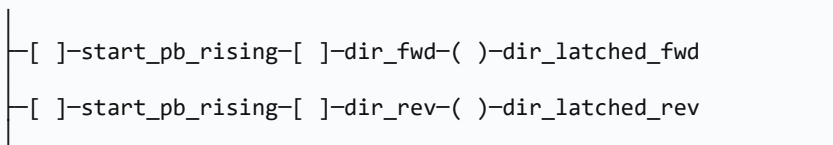
Classic seal-in. `start_pb_rising` is the rising-edge bit of the green PB; `run_Latch` holds itself in once started, as long as `run_ok` stays TRUE.

Rung 3 — Stop on direction change while running

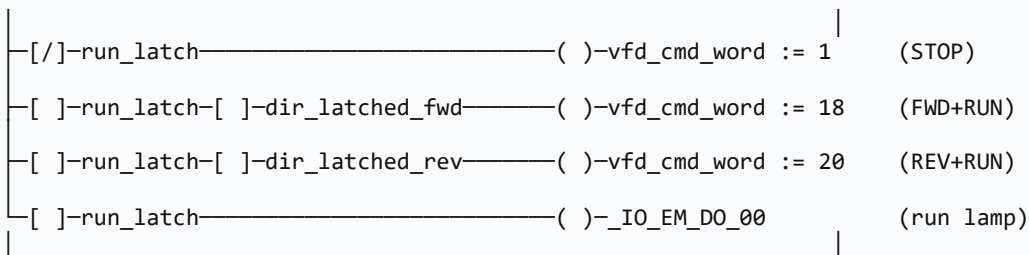


If the direction switch is in a different position than the one captured at start, drop the latch.

Rung 4 — Capture the direction at the moment of start



Rung 5 — Assign the VFD command word and the run lamp



If you see this, the simple version works

Green PB while FWD selector → `vfd_cmd_word` = 18 within one scan; VFD keypad shows commanded frequency within ~2 sec; `_IO_EM_DO_00` lights solid; flipping selector to OFF drops `vfd_cmd_word` to 1 within one scan; flipping selector REV mid-run also drops to 1 and requires a fresh PB press.

4 Why This Becomes a Problem

You wired five rungs. They work. Now imagine the conveyor gets a second motor — say a discharge cross-belt — that needs the same exact behavior: its own green PB, its own selector, its own e-stop (or sharing the main e-stop), its own run lamp, writing to its own VFD's command word over Modbus. To replicate the five rungs:

- Five rungs become ten rungs.
- Every tag in those new five rungs gets a "_2" suffix or its own new name.
- You find a bug in the seal-in logic (say, you realize the dropping-direction rung doesn't account for the "both selector contacts closed = wiring fault" case). You fix it on motor 1's rungs. Two weeks later, motor 2 has the same fault and you spend an hour wondering why it doesn't behave like motor 1.
- The five-rung pattern is now ten rungs; if a third motor appears, fifteen. Each is a copy of the same logic with slightly different tag names. The cognitive load to verify "yes, all three are doing the same thing" is no longer linear.

The pain made concrete

This isn't hypothetical. The deployed Prog2 ST today has a 5-state machine that does roughly the same job, and adding a second conveyor motor would mean copy-pasting the whole `CASE conv_state OF` block with renamed locals. The state-machine approach is more powerful than the flat rungs above (timers, fault states, recovery transitions) but it suffers from the same scaling problem when motor-count grows.

5 The Better Pattern — User-Defined Function Block

A UDFB is a sealed box of logic with labeled input and output terminals. You define it once — name it, declare its inputs, declare its outputs, declare its private locals, write the body. From that point on, every place in the program that needs that behavior just drops the block in and wires the terminals. Inside, the logic runs identically every time. The terminals are the contract: as long as the caller wires the inputs to something sensible, the block does what it promises.

Why this matters: A UDFB is a sealed starter cabinet. You wire field devices to the terminals on the front — start PB, direction switch, e-stop, run lamp, VFD command. Inside is the latching logic, identical in every cabinet. Build one cabinet, install two. Fix a bug in the cabinet design, every cabinet on the floor is now fixed.

The block has **instances**. Each instance is an independent copy of the internal storage (in our case: `RunLatch`, `DirLatch`, `PrevStartPB`). One motor's instance latches separately from the other's, even though both run the same block code. That's the leverage of the pattern.

6 CCW Step-by-Step Walkthrough

Have CCW open with the `MIRA_v5` project loaded. We're going to create one UDFB (`motor_starter`), call it once from a new ladder rung in Prog3 (or a fresh Prog4), and remove the conflicting `vfd_cmd_word :=` assignments from Prog2 ST.

Step 1 — Create the UDFB

1. ☐ Project Organizer → right-click **User-Defined Function Blocks** → **Add** → **New Function Block**.
2. ☐ Language: choose **Structured Text**. (You can pick LD for an inside-the-block ladder version too; ST is shorter for this case and reads more directly.)
3. ☐ Name: `motor_starter` . Press OK.
4. ☐ The block opens with an empty variable list and an empty ST body editor.

Step 2 — Declare the variables (in order — order matters)

In the variable list at the top of the block editor, add these entries. The order you list them determines the order the pins appear when you drop the block onto a calling rung, so group related pins together.

Section	Name	Type	Initial
VAR_INPUT	<code>Enable</code>	BOOL	TRUE
VAR_INPUT	<code>StartPB</code>	BOOL	FALSE
VAR_INPUT	<code>DirFwdSw</code>	BOOL	FALSE
VAR_INPUT	<code>DirRevSw</code>	BOOL	FALSE
VAR_INPUT	<code>EStopOK</code>	BOOL	FALSE
VAR_OUTPUT	<code>VFDCmdWord</code>	INT	1
VAR_OUTPUT	<code>RunLamp</code>	BOOL	FALSE
VAR (local)	<code>RunLatch</code>	BOOL	FALSE
VAR (local)	<code>DirLatch</code>	INT	0
VAR (local)	<code>PrevStartPB</code>	BOOL	FALSE
VAR (local)	<code>StartPBRising</code>	BOOL	FALSE
VAR (local)	<code>DirChangedWhileRunning</code>	BOOL	FALSE
VAR (local)	<code>DropConditions</code>	BOOL	FALSE

VAR_INPUT default initial value — non-obvious gotcha

CCW lets you set an initial value on a `VAR_INPUT` declaration. That initial is only used if the caller leaves the pin unwired. For `Enable` , defaulting to `TRUE` means "if the caller forgets to wire it, the block runs" — which is what you want for a master gate. For everything else, default to `FALSE` so an unwired input never accidentally starts a motor.

Step 3 — Paste the ST body

Click into the ST editor and paste this verbatim. The body is intentionally short — about 30 lines.

```
(* === motor_starter – green PB seal-in with direction-lock + e-stop drop === *)

(* Rising-edge on start pushbutton – prevents stuck-PB auto-restart *)
StartPBRising := StartPB AND NOT PrevStartPB;
PrevStartPB := StartPB;

(* Did the operator flip the selector to a different position while running? *)
DirChangedWhileRunning := RunLatch AND (
    (DirLatch = 1 AND NOT DirFwdSw) OR
    (DirLatch = 2 AND NOT DirRevSw)
);

(* Anything that should drop the seal-in latch *)
DropConditions := (NOT Enable)
    OR (NOT EStopOK)
    OR (NOT DirFwdSw AND NOT DirRevSw) (* selector OFF = stop *)
    OR (DirFwdSw AND DirRevSw) (* both contacts = wiring fault *)
    OR DirChangedWhileRunning;

(* Seal-in state machine *)
IF DropConditions THEN
    RunLatch := FALSE;
    DirLatch := 0;
ELSIF StartPBRising AND NOT RunLatch THEN
    IF DirFwdSw THEN
        RunLatch := TRUE;
        DirLatch := 1;
    ELSIF DirRevSw THEN
        RunLatch := TRUE;
        DirLatch := 2;
    END_IF;
END_IF;

(* Output assignment *)
IF NOT RunLatch THEN
    VFDCmdWord := 1; (* STOP *)
ELSIF DirLatch = 1 THEN
    VFDCmdWord := 18; (* FWD + RUN – bench-verify, see cheatsheet *)
ELSIF DirLatch = 2 THEN
    VFDCmdWord := 20; (* REV + RUN – bench-verify, see cheatsheet *)
ELSE
    VFDCmdWord := 1; (* defensive fallback *)
END_IF;
RunLamp := RunLatch;
```

Step 4 — Build the UDFB

1. ☐ Press **F8** (Build). Look for **0 errors**.
2. ☐ If errors appear, check: did you typo an input name in the ST body vs the variable list? CCW treats UDFB scope as case-insensitive but spelling-sensitive — `DirFwd` ≠ `DirFwdSw`.

If you see "0 errors, 0 warnings"

The UDFB is now a usable building block in your project. Project Organizer → User-Defined Function Blocks → `motor_starter` will show it, and the Toolbox in any program's editor will show it under the project's UDFB section.

Step 5 — Add the calling rung in a new LD program

You can call the UDFB from Prog3 (where the Modbus rungs live), or — cleaner — add a Prog4 specifically for motor control logic. Either works. The walkthrough below assumes Prog4 for clarity.

1. ☐ Project Organizer → Programs → right-click → **Add** → **New LD: Ladder Diagram** → name it `Prog4-MotorControl`.
2. ☐ Set execution order: Prog2 first, Prog3-Modbus second, Prog4-MotorControl third. Properties → Order.
3. ☐ Open Prog4. Drag a contact onto rung 1, type `enable_motor_ctr1` (a global BOOL you'll create; default TRUE).
4. ☐ From Toolbox → User-Defined Function Blocks → drag `motor_starter` onto the rung. Instance name dialog: enter `motor_starter_inst`.
5. ☐ Wire each pin by clicking the pin and typing the tag:
 - `Enable` ← (wired from the contact)
 - `StartPB` ← `_IO_EM_DI_04`
 - `DirFwdSw` ← `dir_fwd`
 - `DirRevSw` ← `dir_rev`
 - `EStopOK` ← `NOT e_stop_active AND NOT estop_wiring_fault` (you may need an inline expression rung; or pre-compute as `e_stop_ok` global in Prog2 ST)
 - `VFDcmdWord` → `vfd_cmd_word`
 - `RunLamp` → `_IO_EM_DO_00`

"EStopOK" can't be a NOT-NOT expression on a ladder pin

CCW won't let you type `NOT e_stop_active AND NOT estop_wiring_fault` directly into the pin field — it expects a single variable. Easiest fix: add one line to Prog2 ST: `e_stop_ok := NOT e_stop_active AND NOT estop_wiring_fault;` and declare `e_stop_ok : BOOL;` as a global. Then wire `EStopOK` ← `e_stop_ok`.

Step 6 — Remove the conflicting `vfd_cmd_word :=` in Prog2 ST

Per the design decision Mike locked when approving this guide: the UDFB drives `vfd_cmd_word` directly. The old state-machine assignments must be removed (or commented out) so both don't write to the same global within one scan.

1. ☐ Open Prog2 in CCW.
2. ☐ Find the `CASE conv_state OF` block.
3. ☐ Comment out (with `(* ... *)`) every line that assigns to `vfd_cmd_word`. There should be three such lines: `vfd_cmd_word := 1;` (in STOP/IDLE/FAULT cases), `vfd_cmd_word := 18;` (in STARTING/RUNNING + `dir_fwd`), `vfd_cmd_word := 20;` (in STARTING/RUNNING + `dir_rev`).

4. ☐ **Leave the rest of the state machine alive.** `conv_state` transitions, `start_timer`, `motor_running`, `item_count` — all keep working. Only the `vfd_cmd_word :=` lines move to the UDFB.
5. ☐ Save Prog2.stf, build the project (F8), download.

Why this matters: If you skip Step 6 and both Prog2 ST and Prog4 LD assign to `vfd_cmd_word` in the same scan, the last writer wins — which depends on program execution order. With order Prog2 → Prog3 → Prog4, the UDFB's value would win for one scan, then Prog2 ST overwrites it on the next, and the motor behaves randomly. Comment the old lines now and never wonder later.

7 Variable Map

Every variable that touches the UDFB and its calling rung. The "Why it belongs here" column is the lesson — read those cells if you want to understand the design, not just copy it.

UDFB external interface (inputs)

Name	Type	Direction	Scope	Purpose	Why it belongs here
Enable	BOOL	Input	Block param	Master gate. Caller passes TRUE in normal operation; can pass FALSE to disable the entire motor without touching the field switches.	Input (not a hardcoded global) because each instance could potentially have its own enable source (HMI button, supervisor PLC, mode-select). Defaulting to TRUE means an unwired pin still runs — preferred for a master gate.
StartPB	BOOL	Input	Block param	The green pushbutton's raw contact value.	Input (not global) because each instance has its own physical pushbutton. Wiring <code>_IO_EM_DI_04</code> inside the block would weld every instance to the same button.
DirFwdSw	BOOL	Input	Block param	FWD position of the direction selector.	Same reasoning — each instance gets its own selector. Wiring to the pre-decoded global <code>dir_fwd</code> rather than the raw <code>_IO_EM_DI_00</code> avoids double-decoding the 3-position logic.
DirRevSw	BOOL	Input	Block param	REV position of the direction selector.	Same as <code>DirFwdSw</code> .
EStopOK	BOOL	Input	Block param	TRUE when the e-stop chain is healthy.	Input (not global) so each instance could in principle be gated by a different safety chain — useful if the discharge motor sits on a different cell with its own zone e-stop. For now the conveyor's main e-stop drives both, but the door is open.

UDFB external interface (outputs)

Name	Type	Direction	Scope	Purpose	Why it belongs here
VFDCmdWord	INT	Output	Block param	The integer value the GS10 command register expects. 1 = STOP; 18 = FWD+RUN; 20 = REV+RUN.	Output (not internal) because the caller's job is to copy this into <code>vfd_cmd_word</code> — which Prog3-Modbus then ships to the drive. If we kept it internal the caller couldn't see it. INT (not BOOL) because the drive's command word is a bit-field integer, not a single bit.
RunLamp	BOOL	Output	Block param	TRUE while the seal-in latch is held.	Output so the caller can wire it to the physical run-indicator output (<code>_IO_EM_DO_00</code>). Keeping it as a block output instead of writing to <code>_IO_EM_DO_00</code> inside the block means a second instance can light a <i>different</i> lamp.

UDFB internal state (locals — each instance has its own)

Name	Type	Direction	Scope	Purpose	Why it belongs here
RunLatch	BOOL	—	Local (UDFB)	The seal-in bit. TRUE = motor is commanded to run.	Local because no caller needs to read it directly — they read RunLamp (which mirrors it). Local also means each instance latches separately: motor A can be running while motor B is stopped.
DirLatch	INT	—	Local (UDFB)	Captured direction at the moment the seal-in was set. 0 = none, 1 = FWD, 2 = REV.	Local because it's the block's private memory of "which way did the operator commit when they pressed the button?" — used by the drop-on-direction-change logic and by the output assignment. INT (not BOOL) because we need three states and "0 = none" is meaningful.
PrevStartPB	BOOL	—	Local (UDFB)	The value of StartPB from the previous scan, used to compute the rising-edge bit.	Local because edge-detect is purely internal bookkeeping. Each instance has its own button, so each tracks its own previous state.
StartPBRising	BOOL	—	Local (UDFB)	Computed each scan as StartPB AND NOT PrevStartPB . TRUE for exactly one scan after the button is pressed.	Local because it's a derived value used only inside the block. Could be inlined into the IF, but giving it a name makes the body 1000% more readable.
DirChangedWhileRunning	BOOL	—	Local (UDFB)	TRUE when the seal-in is set AND the live direction switch differs from DirLatch .	Local; named for readability. Could be inlined into DropConditions , but pulling it out lets you see the "force re-start on direction change" rule on a single line.
DropConditions	BOOL	—	Local (UDFB)	OR of every reason to drop the seal-in latch.	Local; gathering all the drop reasons in one place means the latch IF is short and the drop reasons are documented next to each other rather than scattered.

Calling-program globals the UDFB instance is wired to

Name	Type	Direction	Scope	Purpose	Why it belongs here
motor_starter_inst	motor_starter	—	Instance (Prog4)	The one and only instance of	Declared in Prog4's variable

Name	Type	Direction	Scope	Purpose	Why it belongs here
				<code>motor_starter</code> for the conveyor motor.	list (auto-created when you drop the block onto a rung). Each UDFB instance has its own copy of <i>all</i> the locals listed above — that's how multiple instances stay independent.
<code>enable_motor_ctrl</code>	BOOL	—	Global tag	Master enable for the UDFB. Default TRUE.	Global because the HMI / supervisor logic may want to disable the motor without changing the field switches. Could be local to Prog4 if no one else touches it; global gives future flexibility.
<code>e_stop_ok</code>	BOOL	—	Global tag	Pre-computed <code>NOT e_stop_active AND NOT estop_wiring_fault</code> so it can be wired into the UDFB pin as a single variable.	Global because CCW LD pin assignments don't accept compound boolean expressions. Computing it once in Prog2 ST gives every program (including the UDFB) a clean tag.
<code>_IO_EM_DI_04</code>	BOOL	Input	Physical input	Green PB raw contact.	Physical input — read in the input update phase of the scan, then visible to the

Name	Type	Direction	Scope	Purpose	Why it belongs here
					program as a BOOL.
dir_fwd	BOOL	—	Global tag (computed by Prog2 ST)	Decoded FWD selector position.	Global because Prog2 already decodes it; the UDFB consumes the result rather than re-implementing the 3-position decoding.
dir_rev	BOOL	—	Global tag (computed by Prog2 ST)	Decoded REV selector position.	Same.
vfd_cmd_word	INT	—	Global tag	The command word the Prog3-Modbus sequencer ships to register 0x2000 .	Global because Prog3-Modbus needs to read it (separately from the UDFB writing it). After this refactor the UDFB is the sole writer and Prog3 is the sole reader — a clean one-writer/one-reader pair.
_IO_EM_DO_00	BOOL	Output	Physical output	Green run lamp.	Physical output — written in the output update phase of the scan.

8 Code Walkthrough — chunk by chunk

Chunk 1 — Edge detection on the start pushbutton

```
StartPBRising := StartPB AND NOT PrevStartPB;  
PrevStartPB := StartPB;
```

The first line computes "was the button low on the previous scan AND is it high this scan?" — that's the definition of a rising edge. The second line shifts the current value into the previous-value slot for next scan.

Why this matters: Without rising-edge detection, holding the green PB while the latch is FALSE would re-fire the start every scan as long as the PB stays pressed. With rising-edge, the start only happens once per physical button press. That difference matters when (a) the operator holds the button — you want one start, not 1000 — and (b) the button welds shut from a stray voltage spike — you don't want the motor to auto-restart every time the seal-in tries to drop.

Chunk 2 — Direction-change-while-running detection

```
DirChangedWhileRunning := RunLatch AND (  
    (DirLatch = 1 AND NOT DirFwdSw) OR  
    (DirLatch = 2 AND NOT DirRevSw)  
);
```

If the seal-in is held AND the captured direction was FWD but the FWD switch contact is no longer closed, the operator must have moved the selector. Same logic for REV. Either way: the motor is about to be stopped (the drop happens on the next chunk).

Chunk 3 — Drop conditions

```
DropConditions := (NOT Enable)  
    OR (NOT EStopOK)  
    OR (NOT DirFwdSw AND NOT DirRevSw)  
    OR (DirFwdSw AND DirRevSw)  
    OR DirChangedWhileRunning;
```

Five reasons to drop the latch, OR'd together. Each on its own line so you can read them top-to-bottom: master disable; e-stop unhealthy; selector in OFF position (both contacts open); selector wiring fault (both contacts closed); operator moved the selector mid-run.

Why this matters: Computing `DropConditions` as a single bool before the IF means the latch logic stays short. If you inlined all five reasons into the IF directly, you'd have a tangle of nested boolean operators that's hard to read and hard to extend. The trade-off cost is one extra named local — well worth it.

Chunk 4 — The seal-in state machine

```

IF DropConditions THEN
    RunLatch := FALSE;
    DirLatch := 0;
ELSIF StartPBRising AND NOT RunLatch THEN
    IF DirFwdSw THEN
        RunLatch := TRUE;
        DirLatch := 1;
    ELSIF DirRevSw THEN
        RunLatch := TRUE;
        DirLatch := 2;
    END_IF;
END_IF;

```

Three branches. **If** any drop condition is true → kill the latch and clear the captured direction. **Else if** the button just went high AND we aren't already running → try to latch in whichever direction the selector points; if neither (selector in OFF), don't latch at all. **Otherwise** (no drop, no fresh edge): do nothing — the latch holds its existing state.

Order of branches matters

Drop comes *before* start. If both happened in the same scan (e.g., operator presses the button at the exact moment the e-stop is released), the drop wins and the start request is ignored. That's the safer order — you don't want a marginal scan timing to accidentally start a motor.

Chunk 5 — Output assignment

```

IF NOT RunLatch THEN
    VFDCmdWord := 1;
ELSIF DirLatch = 1 THEN
    VFDCmdWord := 18;
ELSIF DirLatch = 2 THEN
    VFDCmdWord := 20;
ELSE
    VFDCmdWord := 1;
END_IF;
RunLamp := RunLatch;

```

Latch not held → STOP. Latch held in FWD → FWD+RUN. Latch held in REV → REV+RUN. Defensive fallback to STOP if the latch is somehow TRUE but the captured direction is 0 (shouldn't happen because chunk 4 only sets the latch in a valid direction, but the ELSE is cheap insurance against a future code change). Run lamp mirrors the latch.

9 Scan Cycle Explanation

The Micro 820 runs a 5-phase loop every scan (typically ~10 ms): read inputs → execute Prog2 → execute Prog3 → execute Prog4 (our new one) → write outputs. Walk one realistic scenario step-by-step:

Scenario: operator presses Green PB while selector is in FWD position

Scan	What happens inside the UDFB instance	What appears on the wire / lamp
N	StartPB = TRUE, PrevStartPB = FALSE → StartPBRising = TRUE. DirFwdSw = TRUE, DirRevSw = FALSE → no drop	vfd_cmd_word global = 18; _IO_EM_D0_00 goes high at end-

Scan	What happens inside the UDFB instance	What appears on the wire / lamp
	condition. ELSIF branch fires: <code>RunLatch := TRUE</code> , <code>DirLatch := 1</code> . Output: <code>VFDCmdWord := 18</code> , <code>RunLamp := TRUE</code> . Then <code>PrevStartPB := TRUE</code> .	of-scan.
N+1	<code>StartPB</code> still TRUE, <code>PrevStartPB</code> = TRUE → <code>StartPBRising</code> = FALSE. No drop conditions. ELSIF doesn't fire (latch already held). Outputs unchanged.	<code>vfd_cmd_word</code> still 18. Prog3-Modbus's polling sequencer eventually ships an FC06 write of 18 to register <code>0x2000</code> — drive starts ramping the motor up. Run lamp solid green.
N+~50	Operator releases the button. <code>StartPB</code> = FALSE → <code>StartPBRising</code> = FALSE (was FALSE last scan too). Drop conditions: still no e-stop, still no off-selector, no direction change. Latch holds. Outputs unchanged.	Motor still running. Run lamp still solid.
N+~2000	Operator flips selector to REV without stopping first. <code>DirFwdSw</code> = FALSE, <code>DirRevSw</code> = TRUE. <code>DirLatch</code> still = 1 → <code>DirChangedWhileRunning</code> = TRUE → <code>DropConditions</code> = TRUE. IF branch fires: <code>RunLatch := FALSE</code> , <code>DirLatch := 0</code> . Output: <code>VFDCmdWord := 1</code> , <code>RunLamp := FALSE</code> .	<code>vfd_cmd_word</code> drops to 1; drive begins coast-to-stop (or decel-to-stop, depending on GS10 P09.xx settings). Run lamp goes dark within one scan.
N+~2050	Operator presses green PB again, selector now in REV. <code>StartPBRising</code> = TRUE. <code>DirRevSw</code> = TRUE → ELSIF branch sets <code>RunLatch := TRUE</code> , <code>DirLatch := 2</code> . Output: <code>VFDCmdWord := 20</code> .	Motor begins ramping in reverse.

Why this matters: The actual Modbus packet transmit happens in the housekeeping phase at the end of each scan, not at the moment the UDFB writes `vfd_cmd_word` . So you'll see the global update instantly in CCW's variable monitor, but the drive sees the change one Modbus polling cycle later (~500 ms in the deployed sequencer cadence). That's why the keypad takes a beat to display the commanded frequency after you press the button.

10 Testing Procedure

- ☐ Build (F8). 0 errors. (If errors mention `e_stop_ok` undeclared, you skipped Step 5's note — add the global + the Prog2 ST line that computes it.)
- ☐ Download to PLC (USB cable preferred for first download after structure changes).
- ☐ Switch PLC to Run.
- ☐ Go online. Variable Monitor — pin `vfd_cmd_word` , `motor_starter_inst.RunLatch` , `motor_starter_inst.DirLatch` .

Test 1 — Green PB while selector in OFF (center) → no start

✓**You should see:** Press and release green PB with selector centered. `RunLatch` stays FALSE. `vfd_cmd_word` stays 1. `_IO_EM_DO_00` stays dark.

Test 2 — Green PB while selector in FWD → forward start

✓**You should see:** Flip selector to FWD. Press green PB. RunLatch goes TRUE. DirLatch = 1. vfd_cmd_word = 18. _IO_EM_DO_00 goes solid green. Within ~2 sec the VFD keypad shows the commanded frequency.

Test 3 — Selector to OFF while running → stop

✓**You should see:** With motor running forward, flip selector to OFF (center). RunLatch drops to FALSE within one scan. vfd_cmd_word drops to 1. Motor coasts to stop. Run lamp dark.

Test 4 — Direction-change-while-running drops latch (the force-restart behavior)

✓**You should see:** Restart in FWD (Test 2). While running, flip selector directly from FWD to REV (skipping OFF if your switch design allows). RunLatch drops to FALSE. vfd_cmd_word drops to 1. Motor stops. Run lamp dark. Operator must press green PB again to start in REV.

Test 5 — Verify REV direction actually spins motor reverse

✓**You should see:** From stopped state with selector in REV, press green PB. vfd_cmd_word = 20. Within ~2 sec motor spins in the reverse direction. **If motor spins forward, the deployed REV value (20) is wrong on your firmware — see the bench-verify open question in the carry-forward bug block at the top of this doc.**

Test 6 — E-stop drops everything

✓**You should see:** With motor running, press e-stop. e_stop_active goes TRUE, e_stop_ok goes FALSE, RunLatch drops within one scan. vfd_cmd_word = 1. Motor stops. Contactor (_IO_EM_DO_02) drops open via separate e-stop logic.

Test 7 — E-stop release alone does not restart

✓**You should see:** Release e-stop. e_stop_active FALSE, e_stop_ok TRUE. RunLatch stays FALSE — motor does NOT auto-restart. Press green PB to restart.

Test 8 — Stuck-PB protection

✓**You should see:** Simulate stuck PB by force-setting _IO_EM_DI_04 = TRUE in CCW's variable monitor (be careful — this is a real force; release after the test). Motor starts once. Flip selector to OFF (drops latch). Motor stays stopped even though StartPB is still TRUE — the rising-edge prevents auto-restart. Release the force.

Pass / Fail summary

Test	Pass	Fail	Notes
1 — PB in OFF doesn't start	☐	☐	
2 — PB in FWD starts forward	☐	☐	
3 — Selector OFF stops	☐	☐	
4 — Direction change drops latch	☐	☐	
5 — REV actually reverses (BENCH-VERIFY)	☐	☐	If fails: try 34 instead of 20 (see cheatsheet)
6 — E-stop drops everything	☐	☐	
7 — E-stop release alone doesn't restart	☐	☐	

Test	Pass	Fail	Notes
8 — Stuck PB protected by rising-edge	☐	☐	

11 Troubleshooting Guide

Symptom	Likely cause	What to check	Fix
Motor never starts; <code>RunLatch</code> stays FALSE no matter what	One of the drop conditions is permanently TRUE	Variable monitor: <code>Enable</code> , <code>EStopOK</code> , <code>DirFwdSw</code> , <code>DirRevSw</code> . If <code>Enable</code> is FALSE, check <code>enable_motor_ctrl</code> global. If <code>EStopOK</code> is FALSE, check <code>e_stop_active</code> and <code>estop_wiring_fault</code> .	Fix the root drop condition. Most common: forgot to set <code>enable_motor_ctrl := TRUE</code> as the global's initial value.
Motor starts but immediately stops on the same scan	You forgot Step 6 — Prog2 ST is still assigning <code>vfd_cmd_word := 1</code> in the IDLE case, and Prog2 runs before Prog4, so the UDFB's 18 gets overwritten	Open Prog2.stf, search for <code>vfd_cmd_word :=</code> . Should find 0 hits.	Comment out all three lines per Step 6.
Direction change DOESN'T drop the latch — motor reverses live	Either <code>DirLatch</code> isn't being captured at start, or the direction-change detection chunk has a typo	Variable monitor: when motor is running, what is <code>motor_starter_inst.DirLatch</code> ? Should be 1 or 2.	If <code>DirLatch</code> is 0 while <code>RunLatch</code> is TRUE, the ELSIF in chunk 4 didn't fire properly — recheck the rising-edge logic.
Live monitor shows the UDFB body but values look frozen	You're viewing the UDFB definition , not the instance	Project Organizer → Programs → Prog4-MotorControl → expand. Find <code>motor_starter_inst</code> in its local variables. Right-click → Monitor.	Monitor the instance, not the definition. Definition view shows the code template; instance view shows the actual runtime values.
After rebuilding the UDFB, <code>RunLatch</code> is TRUE on first scan even though motor wasn't running before	Some CCW versions retain instance memory across downloads if you didn't fully clear retention	Compare instance value before and after download.	Project → Clear Memory Retention before re-downloading. Or power-cycle the PLC.
Drop on dual-contact-closed (selector wiring fault) never triggers	Your selector is mechanically interlocked so both contacts can't be closed simultaneously — that's a valid design that makes	Check selector datasheet for "make-before-break" vs "break-before-make"	If selector physically can't close both, the check is defensive only — leave it in for documentation but don't expect to ever see it fire.

Symptom	Likely cause	What to check	Fix
	the check unreachable		
Adding a second instance breaks the first one	You wired BOTH instances to the same physical I/O	Inspect both instance call rungs. Each instance's <code>StartPB</code> input must point to a different physical input.	Wire each instance to its own pushbutton, its own selector, its own lamp. Each instance is independent only if its <i>inputs</i> are independent.
Build fails with "undefined identifier <code>e_stop_ok</code> "	You added the global to Prog2 ST's body but forgot to declare it in the global variable list	CCW: Global Variables view. Search for <code>e_stop_ok</code> .	Add <code>e_stop_ok : BOOL;</code> to the global variable list. Rebuild.

12 Mental Model

Why this matters: A UDFB is a sealed starter cabinet. The five labeled terminals on the front of the cabinet — Enable, StartPB, DirFwdSw, DirRevSw, EStopOK — are where you wire the field devices. The two output terminals on the back — VFDCmdWord and RunLamp — are where the cabinet tells the outside world what it has decided to do. Inside is the latching logic, the edge detector, the direction lock. Identical in every cabinet you install. The first time you build one it takes an afternoon; the second cabinet takes ten minutes (drop it on a rung, wire seven pins). The cabinet's internal wiring is the lesson; once you've built one you can build twelve.

13 What You Should Understand Now

- **Why rising-edge is the right trigger for "start".** Level-triggered starts allow stuck buttons to auto-restart the motor — unsafe. Rising-edge requires a fresh physical press for each restart.
- **Why drop conditions are computed before the latch IF.** Centralizing "everything that should stop the motor" into one named bool makes the body readable and makes adding a new drop condition (say, an over-temperature signal) a one-line change.
- **Why each instance has its own `RunLatch`.** The UDFB's locals are per-instance — that's the leverage of the pattern. Motor A and motor B latch independently with the same code.
- **Why `ESTopOK` is an input, not a hardcoded global reference inside the UDFB.** Future flexibility: a second instance could be gated by a different safety chain. Hardcoded globals weld every instance to the same wire.
- **Why `VFDCmdWord` is an INT output, not a BOOL.** The GS10's command word is a bit-field integer (1 / 18 / 20); the UDFB ships the whole integer, not a single bit.
- **Why the direction-change drop is in the drop conditions, not in a separate handler.** Anything that should stop the motor goes in `DropConditions` — one place to read, one place to extend.
- **Why Prog2 ST's `vfd_cmd_word :=` lines had to be commented out.** Two writers to the same global within one scan = whichever ran last wins = unpredictable behavior. One writer is the contract for sanity.

- **Why the live monitor shows different values depending on whether you opened the instance or the definition.** The definition is the code template — its variable values are meaningless. The instance is the runtime data — its values are what the PLC actually sees this scan.

14 Next Step

Two natural next guides, in order:

1. **Conv_Simple_UDFB_DualMotor.html** — instantiate `motor_starter` twice for a hypothetical second conveyor motor. Same UDFB, different I/O wiring (its own PB, its own selector, its own lamp, writing to a separate `vfd_cmd_word_2` global ultimately destined for a second VFD on Modbus slave 2). The lesson is "this is what the UDFB pays back" — one definition, two instances, two independent motors, no copy-paste.
2. **Conv_Simple_UDFB_VFDFeedback.html** — add a `MinFreqOK` input to `motor_starter` that gates the start until the drive reports it's at a stable DC bus voltage (`read_data(3)` from the Prog3-Modbus monitor read). Demonstrates how the UDFB's contract grows as you wire in feedback from the drive — and how a contract change ripples cleanly to every instance.

15 Appendix — tear-off cheat sheet

① Quick reference (rip this page off)

UDFB interface (in declared order)

Pin	Type	Wired to (this conveyor)
Enable	BOOL in	<code>enable_motor_ctrl</code>
StartPB	BOOL in	<code>_IO_EM_DI_04</code>
DirFwdSw	BOOL in	<code>dir_fwd</code>
DirRevSw	BOOL in	<code>dir_rev</code>
EStopOK	BOOL in	<code>e_stop_ok</code>
VFDcmdWord	INT out	<code>vfd_cmd_word</code>
RunLamp	BOOL out	<code>_IO_EM_DO_00</code>

Cmd word values (bench-verify)

STOP	1
FWD + RUN	18
REV + RUN	20

Drop conditions (anything that stops the motor)

- `NOT Enable`

- NOT EStopOK
- Selector OFF (NOT DirFwdSw AND NOT DirRevSw)
- Selector wiring fault (DirFwdSw AND DirRevSw)
- Direction change while running (RunLatch AND captured-dir ≠ live-dir)

Prog2 ST cleanup checklist (one-time)

1. ☐ Add e_stop_ok : BOOL; to global variables
2. ☐ Add e_stop_ok := NOT e_stop_active AND NOT estop_wiring_fault; at top of Prog2 ST body
3. ☐ Comment out all three vfd_cmd_word := lines in the CASE conv_state OF block
4. ☐ Build, download, smoke-test against §10 procedure